

CAN 通信完全指南 (V3) —— 从一帧报文的物理格式层层向上

面向完全不懂 CAN 的读者，也是 slider_upper 项目的权威技术梳理文档。V3 版重构说明：本版彻底调整了讲解顺序——从总线上真实传输的一帧 CAN-FD 报文的物理格式入手，逐段回答“这一段的数据是谁提供的：是硬件自动生成的？是 PCANBasic 驱动填的？还是应用层代码算出来的？”，然后层层向上（PCANBasic.py → can.py → 应用层协议 → 协议文档对照），最后并入完整的《实用报文对照表》（原独立文档 [CAN-报文对照表.md](#) 的全部内容）。所有协议结论均已对照 [KAI执行器通信32字节协议.xlsx](#) 原文逐条核对。

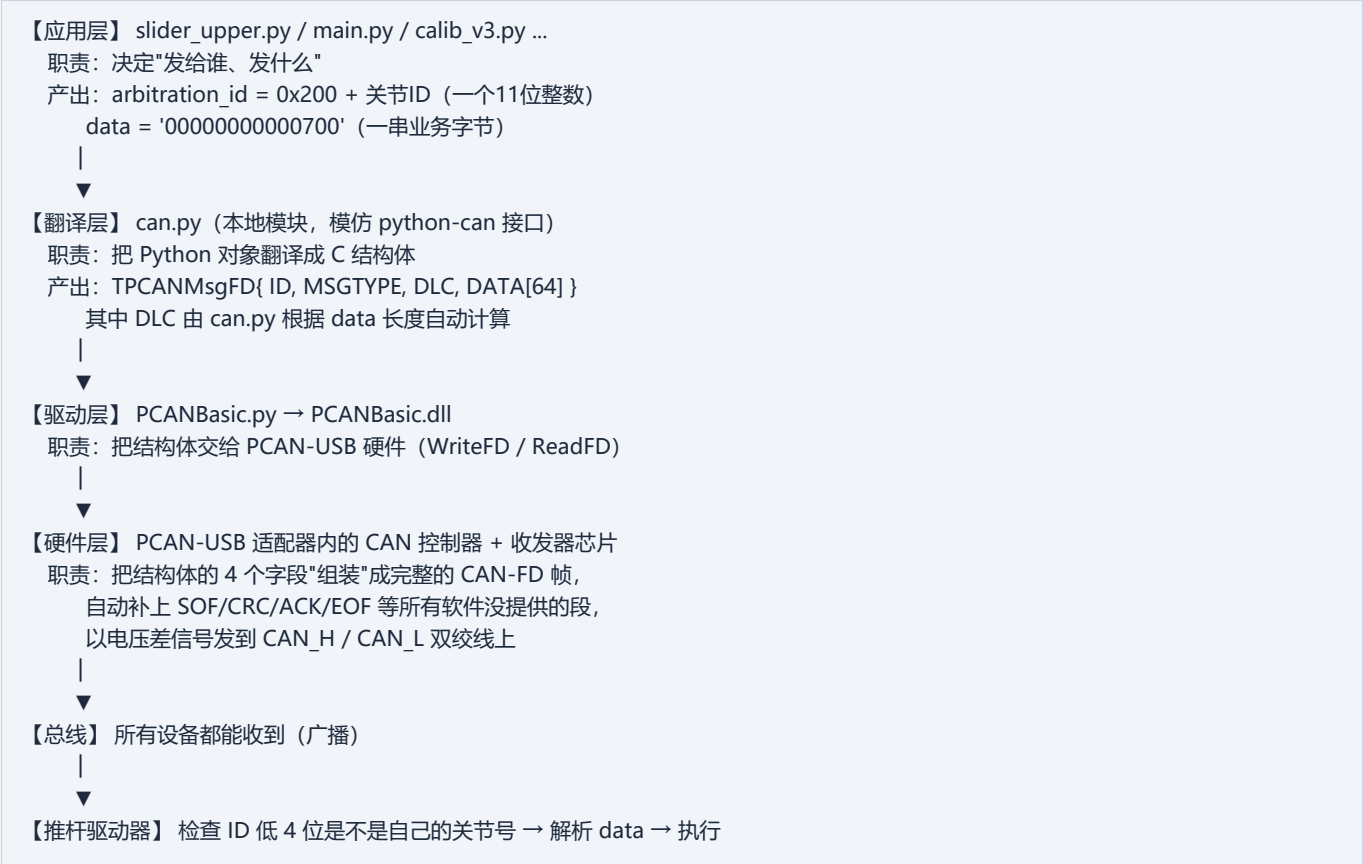
配套代码：[can.py](#)、[PCANBasic.py](#)、[slider_upper.py](#)、[calib_v3.py](#)、[calib_v4.py](#)、[main.py](#)、[position_light.py](#)

目录

- 第1章 总览：一条指令的完整旅程（先看地图）
- 第2章 物理层解剖：一帧 CAN-FD 报文，每一段的数据从哪来
- 第3章 第一层软件：PCANBasic.py 与 TPCANMsgFD——软件世界的“发货单”
- 第4章 第二层软件：can.py——Python 对象与 C 结构体的翻译官
- 第5章 第三层：应用层协议——ID 和 data 的含义从这里开始
- 第6章 协议文档对照：xlsx 是权威，docx 是别家产品线
- 第7章 实用报文完全对照表（18 条，逐字节）
- 第8章 端到端完整实例：使能 → 移动 → 读位置 → 失能
- 第9章 FAQ：曾经真实困惑过的问题
- 附录A DLC 映射表 / 附录B SDO 命令字速查 / 附录C 已知问题待办清单

第1章 总览：一条指令的完整旅程（先看地图）

在钻进细节之前，先把整个体系的“地图”放在这里。后面每一章都是在放大这张图的某一层。



全文只需要记住一个分类法。一帧报文里的每一段数据，来源只有三种：

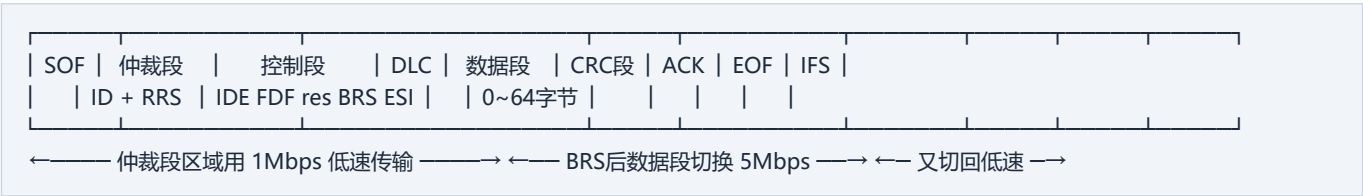
标记	含义	例子
【应用层】	业务代码算出来、拼出来的	ID 的数值、data 的每个字节
【can.py】	翻译层根据应用层输入自动推导的	DLC、MSGTYPE 标志位
【硬件】	驱动/芯片自动生成，任何 Python 代码都碰不到	SOF、CRC、ACK、EOF、仲裁过程

第2章会把一帧报文的每一段逐个贴上这三种标签。

第2章 物理层解剖：一帧 CAN-FD 报文，每一段的数据从哪来

2.1 先看全貌

本项目发送的都是 11 位标准帧、FD 格式、带 BRS 加速 的数据帧。一帧在总线上从头到尾依次包含这些段：



2.2 逐段来源对照表（本章核心）

段	长度	作用	数据从哪来
SOF 帧起始	1 bit	一个显性位，宣告"我要开始发帧了"，全网同步	【硬件】自动生成

段	长度	作用	数据从哪来
标识符 ID	11 bit	双重身份：①总线仲裁的优先级（数值越小越优先）②应用层的"收件地址+消息类型"	【应用层】 <code>0x200+device_id</code> 算出 → 存入 <code>TPCANMsgFD.ID</code> → 硬件原样编码成 11 个比特
RRS	1 bit	经典 CAN 的 RTR 位在 FD 帧里被固定为显性（FD 不支持远程帧）	【硬件】自动生成
IDE	1 bit	0=标准帧(11位ID), 1=扩展帧(29位ID)	【应用层】 <code>is_extended_id=False</code> → 【can.py】不设扩展标志 → 硬件编码为显性
FDF	1 bit	0=经典CAN帧, 1=CAN-FD帧	【应用层】 <code>is_fd=True</code> → 【can.py】设 <code>PCAN_MESSAGE_FD</code> 标志 → 硬件编码
res 保留位	1 bit	协议保留	【硬件】自动生成
BRS	1 bit	1=数据段切换到高波特率（本项目 5Mbps）	【can.py】发送 FD 帧时固定附加 <code>PCAN_MESSAGE_BRS</code> 标志 → 硬件编码
ESI	1 bit	发送节点当前错误状态指示	【硬件】自动生成（由 CAN 控制器错误计数器决定）
DLC	4 bit	数据长度编码（0~15，与真实字节数非线性映射，见附录A）	【can.py】根据 <code>len(data)</code> 自动计算，应用层不用管
数据段 DATA	0~64 字节	真正的业务内容（使能字、目标位置、SDO 指令.....）	【应用层】拼好 hex 字符串 → bytes → 【can.py】逐字节拷入 <code>DATA[64]</code> 数组 → 硬件按 DLC 指定的长度发送
CRC 段	17/21 bit + 计数位	循环冗余校验，接收方用它验证这帧有没有传错	【硬件】发送时自动计算，接收时自动校验，出错自动丢弃并报错误帧
ACK 段	2 bit	总线上任意一个正确收到此帧的节点，在 ACK 槽回填一个显性位作为"签收"	【接收方硬件】自动应答，发送方软件完全无感知
EOF 帧结束	7 bit	7 个连续隐性位标记帧结束	【硬件】自动生成
IFS 帧间隔	3 bit	两帧之间的最小间隙	【硬件】自动保证

一句话总结：软件世界（应用层 + can.py）只提供 4 样东西——ID、DATA、帧类型标志（IDE/FDF/BRS 的意愿）、以及间接决定 DLC 的数据长度；帧里其余所有段（SOF、RRS、res、ESI、CRC、ACK、EOF、IFS），连同"仲裁时逐位比较谁优先"这个动作本身，全部是 CAN 控制器芯片按协议标准自动完成的，任何 Python 代码里都找不到它们。

2.3 波特率是谁定的？

波特率不是帧里的字段，而是通道初始化时的全局配置，一次设定、后续所有帧共用。它由应用层在打开总线时通过 `can.py` 的配置字符串下发：

```
# can.py 第112~114行
# nom_* → 仲裁段 (Nominal) 1 Mbps; data_* → 数据段 (Data) 5 Mbps; 时钟 80MHz
BitrateFD = b'f_clock_mhz=80,nom_brp=2,nom_tseg1=29,nom_tseg2=10,nom_sjw=10,data_brp=1,data_tseg1=11,data_tseg2=4,data_sjw=10'
```

这串参数在 `Bus._init__` 里传给 `InitializeFD()`，由驱动写入硬件的位时序寄存器。之后每一帧的"仲裁段区域跑 1M、BRS 位之后切 5M"都是硬件按这个配置自动执行的。

2.4 实测解剖一帧：「使能 1 号关节」

应用层代码只写了两个值：

```
bus.send(can.Message(
    arbitration_id=0x201,          # 0x200 + 1
    data=bytes.fromhex('00000000000700'), # 7 字节
    is_extended_id=False, is_fd=True))
```

这帧在总线上真实的样子，逐段标注来源：

段	实际值	来源
SOF	显性位 ×1	【硬件】
ID	0x201 = 二进制 01000000001 (11位)	【应用层】0x200+1
RRS	显性	【硬件】
IDE	显性 (标准帧)	【应用层】is_extended_id=False
FDF	隐性 (FD帧)	【应用层】is_fd=True
res	显性	【硬件】
BRS	隐性 (数据段加速→此位之后切换到5Mbps)	【can.py】固定设置
ESI	视节点错误状态	【硬件】
DLC	7 (7字节 ≤8, DLC=字节数)	【can.py】由 len(data)=7 算出
DATA	00 00 00 00 00 07 00	【应用层】hex 字符串
CRC	由前面所有位算出的校验值	【硬件】
ACK	推杆驱动器 (或任何在线节点) 回填显性位	【接收方硬件】
EOF+IFS	隐性位串	【硬件】

看懂这张表，你就已经理解了整个物理层。接下来三章讲软件里那 4 样东西是怎么一层层传下来的。

第3章 第一层软件：PCANBasic.py 与 TPCANMsgFD——软件世界的"发货单"

PCANBasic.py 是 PEAK 厂商官方提供的 Python 绑定，它本身不做任何业务逻辑，只定义了和 PCANBasic.dll 交互所需的常量、结构体和函数封装。

3.1 TPCANMsgFD：软件能控制的全部东西

```
# PCANBasic.py 第395~406行
class TPCANMsgFD(Structure):
    _fields_ = [
        ("ID", c_uint),          # 11/29 位标识符 (32位容器，本项目只用低11位)
        ("MSGTYPE", TPCANMessageType), # 帧类型标志：标准/扩展、CAN/FD、BRS
        ("DLC", c_ubyte),        # 数据长度编码 0~15
        ("DATA", c_ubyte * 64),   # 定长64字节数组
    ]
```

这个结构体就是"软件与硬件的合同"：软件只需要（也只能够）填这 4 个字段，其余帧段一概由硬件代劳。字段与第2章帧段的对应关系：

结构体字段	类型细节	对应帧里的哪一段
ID	c_uint, 4字节容器，有效数据只有低11位 (0~0x7FF)	仲裁段的 11 位标识符
MSGTYPE	位标志组合：PCAN_MESSAGE_FD(FD帧) \	PCAN_MESSAGE_BRS(加速) \
DLC	c_ubyte, 取值 0~15	控制段的 4 位 DLC

结构体字段	类型细节	对应帧里的哪一段
DATA	<code>c_ubyte * 64</code> ，固定长度的 C 数组（不管实际几字节，容器都是64格）	数据段；实际发送多少字节由 DLC 决定，数组尾部没用到的格子是垃圾值、不会被发送

常见误区澄清：ID 是 `c_uint`（4字节容器）不代表“仲裁ID是4字节的数据”——CAN 标准帧的 ID 只有 11 个比特，容器比内容大而已。同理 DATA 是64格容器，使能帧只用了前7格。

3.2 三个关键函数

函数	作用	谁调用
InitializeFD(通道, 波特率串)	打开 PCAN-USB 通道、写入位时序配置	<code>can.py Bus.__init__</code>
WriteFD(通道, TPCANMsgFD)	把一份填好的结构体交给硬件发送	<code>can.py Bus.send</code>
ReadFD(通道)	从硬件接收队列取一帧，返回 (状态码, TPCANMsgFD, 时间戳)	<code>can.py Bus.recv</code>

到这一层为止，软件的职责就结束了。SOF、CRC、ACK、EOF、比特填充、总线仲裁——全部发生在 WriteFD 之后的芯片世界里。

第4章 第二层软件：can.py——Python 对象与 C 结构体的翻译官

can.py 是本项目自己写的本地模块（不是 pip 装的库！），它模仿了业界最流行的 python-can 库的接口（Message/Bus/CanError 这几个类名和 arbitration_id 这个字段名都是在那里沿用的行业惯例，不是本项目发明的新概念）。

千万不要 `pip install python-can`：本项目模块名就叫 can.py，装了公共库会导致 `import can` 指向冲突。

4.1 Message 类：一个类、两条路径

Message 的构造函数身兼两职，靠参数区分：

发送路径：Message(arbitration_id=..., data=...) ← 应用层主动构造，准备发出
接收路径：Message(msgFD=..., timestamp=...) ← Bus.recv 内部构造，还原收到的帧

发送路径做的三件事

```
# can.py 第34~66行（节选）
self.data = data
self.arbitration_id = arbitration_id
dlen = len(data)
self.msgCanMessageFD.ID = arbitration_id    # ① ID 原样搬运，不做任何加工

if dlen <= 8:
    self.msgCanMessageFD.DLC = dlen          # ② 按 CiA 规范算 DLC（见附录A）
elif dlen <= 12:
    self.msgCanMessageFD.DLC = 9
# ..... 13~16→10, 17~20→11, 21~24→12, 25~32→13, 33~48→14, 49~64→15

if is_fd:
    # ③ 组装 MSGTYPE 标志
    self.msgCanMessageFD.MSGTYPE = PCAN_MESSAGE_FD.value | PCAN_MESSAGE_BRS.value

for i in range(dlen):
    # ④ 逐字节拷贝进 C 数组
    self.msgCanMessageFD.DATA[i] = data[i]
```

为什么要“逐字节”拷贝？因为 DATA 是 ctypes 定长数组（`c_ubyte * 64`），Python 的 bytes 对象和它内存布局不兼容，整体切片赋值要求两边长度完全相等（7 ≠ 64 会报错）。所以最稳妥的办法就是循环填前 dlen 格，剩下的格子不动——反正 DLC 已经告诉硬件“只有前7字节有效”，垃圾格子不会被发送。可以想象成一个固

定64格的快递分拣架：你有7件包裹就放前7格，后57格空着无所谓。

接收路径 (msgFD 是什么)

msgFD 就是 驱动从硬件真实收到的一帧，包在 TPCANMsgFD 结构体里。Bus.recv 调用 ReadFD 拿到它之后，交给 Message(msgFD=...) 做反向翻译：

```
# can.py 第68~92行 (节选)
def __init_msgFD(self, msgFD, timestamp):
    self.msgCanMessageFD = msgFD
    dlc = msgFD.DLC
    if dlc <= 8: data = msgFD.DATA[0:dlc] # 按 DLC 反查真实长度 (与发送时对称)
    elif dlc == 13: data = msgFD.DATA[0:32] # 推杆 32 字节状态报文走这档
    elif dlc == 14: data = msgFD.DATA[0:48]
    # .....
    self.data = bytes(data) # C 数组 → Python bytes
    self.arbitration_id = msgFD.ID # ID 原样搬运回来
    self.timestamp = timestamp.value
```

一句话：发送方向，你造结构体给驱动去发；接收方向，驱动造好结构体给你去读。同一种结构体，只是"谁负责填"不同。

4.2 Bus 类：开门、发货、收货、关门

方法	做什么
<code>__init__</code>	加载 DLL → InitializeFD(PCAN_USBBUS1, BitrateFD)，失败抛 CanError
<code>send(message)</code>	WriteFD(通道, message.msgCanMessageFD)
<code>recv(waittime)</code>	循环调 ReadFD 轮询，队列空则每 10μs 重试直到超时；收到则返回 Message(msgFD=...)
<code>shutdown()</code>	释放通道

4.3 can.py 的"无知"是设计出来的

can.py 从头到尾不知道什么是关节ID、什么是使能、什么是 SDO 的 index/subindex。它对 arbitration_id 唯一的操作是"原样搬进结构体"，对 data 唯一的操作是"算长度、拷字节"。所有业务含义都属于上一层——这就是为什么你在 can.py 里找不到任何协议知识：不是它漏了，是分层设计本该如此。

第5章 第三层：应用层协议——ID 和 data 的含义从这里开始

从这一章开始，才轮到"推杆厂商的协议"登场。应用层要回答两个问题：ID 填什么数？data 填什么字节？

5.1 仲裁 ID 的公式：基础通道号 + 关节 ID

完整 11 位 ID = 基础 COB-ID (表示消息类型) + 关节 ID (1~10, 表示哪台设备)

本项目用到的全部通道：

基础 ID	方向	用途	项目是否实际使用
0x180 + id	驱动 → 主机	CiA402 状态字 (故障位)	√ 使用
0x200 + id	主机 → 驱动	简化控制帧：使能/失能/位置/速度/电流	√ 项目唯一的运动控制通道
0x480 + id	驱动 → 主机	状态反馈 TxPDO4 (32/48字节)	√ 使用

基础 ID	方向	用途	项目是否实际使用
0x500 + id	主机 → 驱动	PVT 力位混合控制 (RxPDO4)	× xlsx 有文档但代码未实现
0x580 + id	驱动 → 主机	SDO 响应	√ 使用
0x600 + id	主机 → 驱动	SDO 配置请求	√ 使用

这套编号不是拍脑袋定的，是 CANopen 标准的"预定义连接集" (Predefined Connection Set) 惯例：0x180=TxPDO1、0x200=RxPDO1、0x480=TxPDO4、0x500=RxPDO4、0x580/0x600=SDO应答/请求。

为什么要这样设计，而不是每台设备随便分配一个固定 ID？三个理由：

1. 扩展方便：新增设备只需给它配一个空闲关节号（写在设备自己的 0x2001:01 对象里），不需要维护全局 ID 映射表。
2. 固件实现极简：驱动器开机读一次自己的节点号，`my_control_id = 0x200 + node_id`，一行加法搞定监听地址。
3. 优先级天然合理：CAN 规定 ID 越小优先级越高。控制命令 (0x2xx) 基础值比 SDO 配置 (0x6xx) 小，总线拥挤时控制帧自动优先——这是有意的分类。

从收到的 ID 反解关节号：取低 4 位。

```
# slider_upper.py 第377~379行（设备扫描）
aid = msg.arbitration_id      # 例如 0x483
joint_id = aid & 0x000F      # 0x483 & 0xF = 3 → 3号关节的反馈
```

注意：发送时用的 `device_id` 来自 GUI 下拉框（用户选的，`slider_upper.py` 第41行 `tk.IntVar(value=1)`），不是从反馈报文实时取的；反馈 ID 的低4位只用于"扫描发现总线上有哪些设备在线"。

5.2 data 的两大家族：这是全项目最容易混淆的地方

同一个 `data` 字段，走不同通道时格式完全不同，千万不要拿 A 家族的格式去套 B 家族：

家族一：自定义控制帧 (`0x200+id` 专用，7 字节定长)

```
[0]  [1:3]  [3:5]  [5:7]
模式字节  目标值(2B小端)  时长ms(2B小端)  控制字(2B小端)
```

模式字节	含义	控制字常见取值
00	无运动目标（纯使能/失能）	0x0007使能 / 0x0004失能
01	位置模式	0x000F（使能+运行）
03	速度模式	0x000F
04	电流模式	0x000F
06	六步换相（调试）	0x000F

重要：这套切分规则的出处是"代码本身"——它硬编码在 `main.py/calib_v3.py` 等文件里，`KAI执行器通信32字节协议.xlsx` 并没有文档化这个格式（xlsx 描述的运动控制走的是另一套 `0x2011+0x500` 的方案，见 6.3 节）。上面的字段含义是通过并排对比多处代码调用反推出来的规律，控制字取值风格与 CiA402 Controlword 一致（0x0007=Switch On，0x000F=Enable Operation）。

家族二：标准 CANopen SDO (`0x600+id` 请求 / `0x580+id` 响应，8 字节定长)

```
[0]  [1:3]  [3]  [4:8]
命令字cs index(2B小端) subindex 数据(按cs决定几字节有效，右侧补0)
```

命令字 cs 的取值见附录B。xlsx 里的 index/subindex 就编码在这里的 data 字节里，不在仲裁 ID 里——这是"xlsx 有 index/subindex、而 can.py 里找不到"这个困惑的答案：index/subindex 是家族二 data 内部的字段，且只有走 SDO 通道的报文才有。

用真实代码验证解码规则（能和 xlsx 对上，证明规则正确）：

```
main.py heart() 发送 "2B171000" + 周期 + "0000"
拆解: cs=2B(写2字节) index=0x1017 subindex=0x00
xlsx"上位机发送"sheet: "关闭心跳 = index 0x1017 sub 0x00" → 完全一致

slider_upper.py 校准发送 '2B01200500000000'
拆解: cs=2B index=0x2001 subindex=0x05 数据=0
xlsx"SDO"sheet: 0x2001 sub5 = PosOffset, INT16, rw, 单位10um → 完全一致
```

反馈通道的 data (接收方向)

- 0x480+id: 32/48 字节定长表, 按字节偏移直接取值 (字段表见 7.14 节), 格式来自 xlsx"上位机接收"sheet, 代码与之逐字节一致。
- 0x180+id: 前 2 字节是 CiA402 状态字 (0x6041), 按位解析故障。
- 0x580+id: 家族二的响应格式。

5.3 字节序与 hex 字符串

项目里所有多字节数值都是小端 (Little-Endian) : 整数 1234 (0x04D2) 在报文里存为 D2 04 (低字节在前) 。

```
(1234).to_bytes(2, byteorder='little', signed=True).hex()      # 编码 → 'd204'
int.from_bytes(bytes.fromhex('d204'), 'little', signed=True)  # 解码 → 1234
```

大量使用 hex 字符串 (bytes.fromhex('2B012005...')) 是因为协议文档里就是这么写的, 方便照抄核对。

5.4 消除"data1 / data2"错觉: 从头到尾只有一份数据

- ① 应用层 hex 字符串 '00000000000700'
| bytes.fromhex
- ② Python bytes 对象 b'\x00\x00\x00\x00\x00\x07\x00' (7字节)
| can.py 逐字节拷贝
- ③ C 结构体 DATA[64] 数组 前7格 = 同样的7个字节, 后57格垃圾值 (不发送)
| WriteFD → 硬件
- ④ 总线电压信号 还是这7个字节, 一位一位串行发出
|
- ⑤ 驱动器固件解析 识别控制字 0x0007 → 执行使能

从②到④, 字节数值一次都没变过, 变的只是"容器": Python bytes → C 数组 → 电信号。不存在"两份不同的 data"。

第6章 协议文档对照: xlsx 是权威, docx 是别家产品线

6.1 两份文档不是一回事!

文件	产品线	定位
KAI执行器通信32字节协议.xlsx	推杆 (直线执行器, 单位 m/N)	本项目权威协议文档
KAIBOT 关节SDO对象字典 (1).docx	关节 (旋转模组, 单位 rad/Nm)	同厂商另一条产品线的字典, 只能当风格参考

两者共用 KAIBOT/KAI 家的底层框架、索引风格接近, 但同一索引含义可能完全不同:

索引	docx (关节) 里的含义	xlsx (推杆) 里的含义
0x2001	(没有这个索引)	Device Info: sub1=设备ID, sub4=丝杆导程, sub5=位置offset
0x2002	故障相关 (sub2=故障掩码)	推力传感器: sub1=拉压力校零, sub3=拉压力系数

索引	docx (关节) 里的含义	xlsx (推杆) 里的含义
0x2024	电机电气/机械偏移校准	(没有这个索引)

查协议字段时，本项目一律以 xlsx 为准。

6.2 xlsx 里有什么 (5 个有效 sheet)

Sheet	内容
上位机发送	SDO 配置指令 (0x600+id) + PVT 控制指令 (0x500+id, index 0x1603)
上位机接收	TxPDO4 状态反馈 32 字节逐字节字段表 (0x480+id)
错误码	DriveResp_Errorcode 完整错误码表 (0x1xxx电压/0x2xxx电流/0x3xxx温度/...)
状态码	驱动状态字位定义 (0x20xx待机/0x40xx伺服/0x41xx PVT/0x4Fxx下伺服)
SDO	完整对象字典 (0x2001~0x200B、0x6041~0x6044、0x6060、0x607D、0xFF01)

6.3 最重要的发现：代码用的控制方式 ≠ xlsx 描述的控制方式

对比项	xlsx 描述的"完整版"协议	代码实际使用的方式
使能/失能	SDO 写 0x2011:01 (0x4000上伺服 / 0x4F00下伺服 / 0x2000待机 / 0x0035清故障)	0x200+id 控制帧, 控制字 0x0007/0x0004
运动目标下发	0x500+id PVT 帧 (index 0x1603), 可同时带位置+速度+力矩+Kp+Kd 做力位混合	0x200+id 控制帧, 单一目标 (位置或速度或电流)
归零	0x2024:07 写目标 + 回读确认 + 0x2011:01=0x0032 应用	SDO 写 0x2001:05 (PosOffset) 的 offset 补偿方案

全项目 grep 不到 0x2011、0x500、0x1603 任何一处——代码走的是一套 xlsx 未文档化的简化格式，xlsx 描述的完整状态机+PVT 方案完全没有被实现。两种解释：驱动固件向下兼容两套指令（代码选了简单的老方式）；或 xlsx 是更新版协议、代码没跟进。目前工具能稳定工作，说明固件确实同时接受这两套。

6.4 xlsx 与代码对照的完整结论

功能	核对结果
offset 校准 (0x2001:05)、改ID (0x2001:01)、力标零 (0x2002:01)、周期上报 (0x1803:05)、心跳 (0x1017:00)	√ 代码与 xlsx 完全一致
0x480 反馈 32 字节字段表	√ 逐字节完全一致
使能/失能/位置/速度/电流控制 (0x200 通道)	△ 能用, 但 xlsx 未文档化该格式
PVT 力位混合 (0x500)、伺服状态机 (0x2011)、清故障	× xlsx 有、代码未实现
main.py 的 Statusword6041 故障文字表	× 已过时, 与 xlsx 的 0x6041 31位定义对不上 (如代码bit6="位置超限", xlsx bit6="C相电流过大")
xlsx 内部矛盾	"上位机发送"sheet 说设置ID是 0x2033:04, "SDO"sheet 说是 0x2001:01; 代码用后者且实测有效, 前者疑似从关节产品线混入的笔误

第7章 实用报文完全对照表 (18 条, 逐字节)

本章收录代码里真正会出现在总线上的全部报文 (逐个 grep 全项目 .py 文件核对)。每条给出: 仲裁ID → data 逐字节 → xlsx 对应位置 → 代码出处。标记: √ = 已用 xlsx 验证一致; △ = 能用但 xlsx 未文档化 (自定义); 标准 = 通用 CANopen 格式。

7.0 速查总表

#	报文	方向	仲裁ID	data (hex)	xlsx对应	状态
1	使能	发	0x200+id	00 00 00 00 00 07 00	无	△自定义
2	失能	发	0x200+id	00 00 00 00 00 04 00	无	△自定义
3	位置控制	发	0x200+id	01+目标(2B)+时长(2B)+0F00	无 (xlsx的位置控制走0x500)	△自定义
4	速度控制	发	0x200+id	03+目标(2B)+时长(2B)+0F00	同上	△自定义
5	电流控制	发	0x200+id	04+目标(2B)+时长(2B)+0F00	同上	△自定义
6	六步换相	发	0x200+id	06+目标(2B)+时长(2B)+0F00	同上	△自定义
7	设置心跳周期	发	0x600+id	2B 17 10 00+周期(2B)+0000	上位机发送/0x10 17:00	√
8	借心跳读状态	发	0x600+id	2B 17 10 00 01 00 00 00	同上 (用法是代码技巧)	√索引/△用法
9	设置周期上报	发	0x600+id	2B 03 18 05+周期(2B)+0000	上位机发送/0x18 03:05	√
10	offset清零	发	0x600+id	2B 01 20 05 00 00 00 00	SDO/0x2001:05 PosOffset	√
11	offset写补偿	发	0x600+id	2B 01 20 05+补偿(2B)+0000	同上	√
12	力传感器标零	发	0x600+id	23 02 20 01 00 00 00 00	SDO/0x2002:01 calibration	√
13	写新设备ID	发	0x600+id	2F 01 20 01+新ID(1B)+000000	SDO/0x2001:01 Device ID	√
14	状态反馈TxPDO4	收	0x480+id	32或48字节定长	上位机接收sheet 逐字节表	√
15	CiA402状态字	收	0x180+id	2字节+	SDO/0x6041位定义	√
16	SDO写响应	收	0x580+id	60+index+sub+...	标准SDO	标准
17	SDO读响应	收	0x580+id	43/47/4B/4F+index+sub+data	标准SDO	标准
18	SDO失败响应	收	0x580+id	80+...	标准SDO abort	标准

7.1 使能 △

- 仲裁ID: 0x200 + 关节ID; data: 00 00 00 00 00 07 00 (7字节)

字节	含义	本条取值
[0]	模式字节	00 (无运动目标, 纯状态切换)
[1:3]	目标值(2B小端)	00 00 (未使用)
[3:5]	时长ms(2B小端)	00 00 (未使用)
[5:7]	控制字(2B小端)	07 00 → 0x0007

xlsx 对照: xlsx 描述的"上伺服"是 SDO 写 0x2011:01=0x4000, 与本条不是同一套指令; 控制字 0x0007 风格类似 CiA402"Switch On", xlsx 未文档化。

代码出处: [calib_v3.py](#) 66行 / [calib_v4.py](#) 31行 / [main.py](#) 333、523行 ([is_enable](#)) / [slider_upper.py](#) 608、752、796行 (内联) / [position_light.py](#) 16行。

```
# calib_v3.py 第57~68行
def is_enable(sign=0):
    if sign == 1:
        send_message(bus, 0x200, '000000000000700') # 使能
    else:
        send_message(bus, 0x200, '000000000000400') # 下使能
```

7.2 失能 [△]

- 仲裁ID: [0x200](#) + [关节ID](#); data: [00 00 00 00 00 04 00](#); 控制字 [0x0004](#)。
- 字节结构与 7.1 完全相同, 仅控制字不同。
- 代码出处: [calib_v3.py](#) 68行 / [calib_v4.py](#) 33行 / [main.py](#) 335、525、538行 / [slider_upper.py](#) 560、640、658、824、910行 / [position_light.py](#) 18行。

7.3 位置控制 [△]

- 仲裁ID: [0x200](#) + [关节ID](#); data: [01](#) + 目标位置(2B小端) + 时长ms(2B小端) + [0F00](#)。

字节	含义	单位
[0]	模式= 01 (位置模式)	固定
[1:3]	目标位置 int16 小端	0.01mm (1234=12.34mm)
[3:5]	运行时长 int16 小端	1ms
[5:7]	控制字 0F 00=0x000F	使能+运行固定尾部

举例: 目标 12.34mm、2000ms → [01 D2 04 D0 07 0F 00](#)。前提: 必须先发使能。

xlsx 对照: xlsx 的正规位置控制走 [0x500+id](#) PVT 帧 (可带 Kp/Kd 力位混合), 本条为自定义简化版, xlsx 未文档化。

```
# main.py 第348~356行
def positionControl(target_position, moveDuration):
    target_position = int(target_position).to_bytes(2, byteorder="little", signed=True).hex()
    moveDuration = int(moveDuration).to_bytes(2, byteorder="little", signed=True).hex()
    data = '01' + target_position + moveDuration + '0F00'
    send_message(bus, 0x200, data)
```

代码出处: [main.py](#) 348-356行 / [position_light.py](#) 24-28行 / [slider_upper.py](#) 612-618、648-653行 (GUI 闭环移动中反复发送校正)。

7.4 速度控制 [△]

- data: [03](#) + 目标速度(2B小端) + 时长(2B小端) + [0F00](#); 结构同 7.3, 模式字节改 [03](#)。
- 代码出处: 仅 [main.py](#) 338-346行 ([speedControl](#), 交互命令 [s](#))。

7.5 电流控制 [△]

- data: [04](#) + 目标电流(2B小端) + 时长(2B小端) + [0F00](#)。
- 代码出处: 仅 [main.py](#) 358-367行 ([currentControl](#), 交互命令 [c](#))。

7.6 六步换相 (调试用) [△]

- data: [06](#) + 参数(2B小端) + 时长(2B小端) + [0F00](#)。

- 代码出处：仅 `main.py` 369-377行 (`fc`) 。唯一在 `xlsx` 任何 `sheet` 都找不到对应说明的模式字节，纯调试/工厂测试用途。

7.7 设置/关闭心跳周期 ✓

- 仲裁ID: `0x600 + 关节ID`; data: `2B 17 10 00 + 周期ms(2B小端) + 00 00`。

字节	含义	取值
[0]	cs	<code>0x2B</code> (写2字节)
[1:3]	index (小端)	<code>0x1017</code>
[3]	subindex	<code>0x00</code>
[4:6]	数据	心跳周期ms, 0=关闭
[6:8]	填充	<code>00 00</code>

`xlsx` 对照: "上位机发送"sheet"关闭心跳"= index`0x1017` sub`0x00`, 完全一致 (`0x1017` 同时也是 CANopen 标准对象 `Producer Heartbeat Time`) 。

代码出处：仅 `main.py` 322-329行 (`heart`) , `test()` 里 `heart(0)` 关闭心跳。

7.8 借心跳触发一次状态读取 (calib_v4 专用技巧) ✓索引/△用法

- 发 `0x600+id: 2B 17 10 00 01 00 00 00` (把心跳周期临时设为 1ms) , 随即在 `0x180+id` 上等驱动器广播的状态字——借"改心跳周期"的副作用实现"主动读状态", 而不是走标准 SDO 读请求 (cs=`0x40`) 。
- 隐患: 函数结束后没把周期改回 0, 多次调用会让驱动器一直保持 1ms 心跳, 增加总线负载。此用法 `xlsx` 未描述, 是代码自己的技巧。
- 代码出处: 仅 `calib_v4.py` 57-65行 (`read_status`) 。

7.9 设置/关闭周期状态上报 ✓

- 仲裁ID: `0x600 + 关节ID`; data: `2B 03 18 05 + 周期ms(2B小端) + 00 00`。

字节	含义	取值
[0]	cs	<code>0x2B</code>
[1:3]	index	<code>0x1803</code> (TPDO4 通信参数)
[3]	subindex	<code>0x05</code> (Event Timer, 周期上报间隔)
[4:6]	数据	项目固定 <code>0x0020=32ms</code> ; <code>main.py</code> <code>is_report()</code> 可传 1 或 0

`xlsx` 对照: "上位机发送"sheet"开启上报"= index`0x1803` sub`0x05` (示例 2ms/500Hz) , 格式完全一致, 项目取值 32ms 只是配置不同。

代码出处 (全项目出现频率最高的报文, 读位置/状态前都先发一次) : `calib_v3.py` 131-138行 / `calib_v4.py` 118行 / `main.py` 313-319行 (`is_report`) / `slider_upper.py` 373、414、452、626、664行。

7.10 offset 清零 ✓

- 仲裁ID: `0x600 + 关节ID`; data: `2B 01 20 05 00 00 00 00`。
- 拆解: cs=`2B`, index=`0x2001`, subindex=`0x05`, 数据=0。
- `xlsx` 对照: "SDO"sheet `0x2001` sub5 = PosOffset, INT16, rw, 单位 10um (=0.01mm) , 完全一致。
- 笔误提醒: `calib_v3.py` 第118行注释写"索引|`0x2005`子索引|`0x01`"是反的, 正确为索引|`0x2001`子索引|`0x05`。
- 代码出处: `calib_v3.py` 117-126行 / `calib_v4.py` 114行 / `slider_upper.py` 759行。

7.11 offset 写入补偿值 ✓

- data: 2B 01 20 05 + 补偿量(2B小端, 有符号) + 00 00; 补偿量 = 用户实测真实位置 - 反馈位置 (单位 0.01mm=10um, 与 xlsx 一致)。
- 代码出处: [calib_v3.py](#) 164-168行 / [calib_v4.py](#) 144-150行 (带5次重试) / [slider_upper.py](#) 792-801行 (GUI 版带重试)。

```
# calib_v3.py 第164~168行
offset_position = int(offset_position).to_bytes(2, byteorder='little', signed=True).hex()
data = '2B012005' + offset_position + '0000'
bus.send(can.Message(arbitration_id=arbi_id, data=bytes.fromhex(data), is_extended_id=False, is_fd=True))
```

7.12 力/推力传感器标零 ✓

- 仲裁ID: 0x600 + 关节ID; data: 23 02 20 01 00 00 00 00。
- 拆解: cs=23 (写4字节), index=0x2002, subindex=0x01, 数据=0。
- xlsx 对照: "SDO"sheet 0x2002 sub1 = calibration, INT32, rw, "拉压力校零", 完全一致。
- 代码出处: 仅 [slider_upper.py](#) 827-835行 ([_force_zero_calib](#)), CLI 脚本无此功能。

7.13 写入新设备 ID ✓

- 发 0x600+当前ID: 2F 01 20 01 + 新ID(1B) + 00 00
00; 拆解: cs=2F (写1字节), index=0x2001, subindex=0x01。
- 收 0x580+当前ID: 期望前4字节 60 01 20 01 (写成功 + index/subindex 回显) ——响应回显与请求完全对应, 反向验证了整套解码规则。
- 生效需重新上电 (xlsx 另有 0x2001:02 Reboot 对象, 代码未用, 提示用户手动断电)。
- xlsx 对照: "SDO"sheet 0x2001 sub1 = Device ID, rw, 完全一致。注意 xlsx 内部矛盾: "上位机发送"sheet 写的 0x2033:04 与此不符, 代码采用 0x2001:01 且实测有效。
- 代码出处: 仅 [slider_upper.py](#) 837-866行 ([_write_new_id](#))。

7.14 状态反馈 TxPDO4 (接收) ✓

- 仲裁ID: 0x480 + 关节ID; 32字节 (DLC=13) 或48字节 (DLC=14); 驱动器按 7.9 配置的周期自动发送。
- 32 字节完整字段表 ([main.py ParseData\(\)](#) 与 xlsx"上位机接收"sheet 逐字节一致):

字节偏移	字段 (xlsx命名)	类型	单位 (推杆)
[0:2]	DriveResp_Status	uint16小端	状态码
[2:4]	DriveResp_PosReal	int16小端	0.00001 m (即0.01mm整数)
[4:6]	DriveResp_VelReal	int16小端	0.0001 m/s
[6:8]	DriveResp_TorqReal	int16小端	1 N
[8:10]	DriveResp_Errorcode	uint16小端	查"错误码"sheet
[10]	InstantFeedback_Ia	int8	1 A
[11]	InstantFeedback_Ib	int8	1 A
[12]	InstantFeedback_Ic	int8	1 A
[13]	InstantFeedback_Udc	int8	1 V
[14:16]	InstantFeedback_Idc	int16小端	0.01 A
[16:18]	InstantFeedback_Id	int16小端	0.01 A
[18:20]	InstantFeedback_Iq	int16小端	0.01 A
[20:22]	InstantFeedback_IdRef	int16小端	0.01 A
[22:24]	InstantFeedback_IqRef	int16小端	0.01 A

字节偏移	字段 (xlsx命名)	类型	单位 (推杆)
[24:26]	InstantFeedback_motorIsLimit	int16小端	0.01 A
[26:28]	InstantFeedback_TorqRef	int16小端	1 N
[28]	InstantFeedback_MosTemperMax	int8	°C
[29]	InstantFeedback_MotorTemperMax	int8	°C
[30:32]	DriveResp_DataIndex	uint16小端	回显 DataIndex

代码出处: [main.py](#) 99-194行 ([ParseData](#), 兼容32/48字节) / [slider_upper.py](#) 440-459行起 ([_update_status_display](#)) 及 416、628、666行 (只取 [data\[2:4\]](#) 位置字段) / [calib_v3.py](#) 154行、[calib_v4.py](#) 75行 (只取位置)。大部分代码只用 [0:10] 这10个字节, [10:32] 的电流/温度细节主要在 [main.py](#) 完整解析里用。

7.15 CiA402 状态字 (接收) ✓

- 仲裁ID: [0x180](#) + 关节ID; [0:2] = 状态字 uint16 小端。
- xlsx 对照: "SDO"sheet [0x6041](#) Statusword, 31 个故障位 (节选): bit2/3 电压过低/过高; bit6/7/8 C/B/A相电流过大; bit9/10/11 相电流偏移校准错误; bit12/13 电机温度警告; bit16/17 电机温度过高; bit22/23 位置后向/前向超限; bit26/27 编码器故障/通讯异常; bit28/29/30 驱动寄存器复位/驱动故障/驱动通信异常; bit31 CANFD PDO 中断。
- 代码解析已过时: [main.py](#) 顶部硬编码的 [Statusword6041](#) 文字表 (13项) 与 xlsx 31位定义对不上 (如代码 bit6="位置超限", xlsx bit6="C相电流过大"), 不要直接信旧表打印的文字。
- 代码出处: [main.py](#) 247-265行 / [calib_v4.py](#) 61-64行 (只取数值不做位解析)。

7.16~7.18 SDO 响应 (接收) 标准

- 仲裁ID: [0x580](#) + 关节ID; 标准 CANopen SDO 响应, 与请求字节结构对称:

类型	[0]cs	[1:3]index	[3]sub	[4:8]数据
写成功	0x60	回显	回显	—
读成功(4/3/2/1字节)	0x43/0x47/0x4B/0x4F	回显	回显	有效数据
失败(abort)	0x80	回显	回显	abort错误码

代码出处: [main.py](#) 266-279行 (cs 取值判断与上表一致) / [calib_v4.py](#) 153-166行 (识别 [0x4F/0x4B](#) + 读 [data\[4:6\]](#) 确认) / [slider_upper.py](#) 859-861行 (匹配固定前4字节 [60012001](#) 判断改ID成功)。

第8章 端到端完整实例: 使能 → 移动 → 读位置 → 失能

场景: 关节 ID=1, 目标 50.00mm (内部值 5000), 移动 1500ms。每步标注三层各自做的事。

步骤0 开总线

```
import can
bus = can.Bus()
# 应用层: 发起; can.py: 把 BitrateFD 配置串传给 InitializeFD; 硬件: 写入位时序寄存器
```

步骤1 使能

```
bus.send(can.Message(arbitration_id=0x200+1, data=bytes.fromhex('00000000000700'),
                    is_extended_id=False, is_fd=True))
# 应用层：算 ID=0x201、拼7字节data
# can.py: DLC=7、MSGTYPE=FD|BRS、字节拷入DATA[64]
# 硬件：补 SOF/CRC/ACK/EOF, 1M 发仲裁段、5M 发数据段
# 驱动器：ID低4位=1是自己 → 控制字0x0007 → 上电使能
```

步骤2 位置指令

```
payload = '01' + (5000).to_bytes(2,'little',signed=True).hex() \
          + (1500).to_bytes(2,'little',signed=True).hex() + '0F00'
# payload = '018813dc050f00'
bus.send(can.Message(arbitration_id=0x201, data=bytes.fromhex(payload),
                    is_extended_id=False, is_fd=True))
# 驱动器：模式01=位置模式，目标5000×0.01mm=50mm，1500ms 内闭环走到
```

步骤3 开周期上报（SDO）并读反馈

```
bus.send(can.Message(arbitration_id=0x601, data=bytes.fromhex('2B03180520000000'),
                    is_extended_id=False, is_fd=True))
# SDO 写 0x1803:05 = 32 → 驱动器每32ms自动发一帧 0x481

msg = bus.recv(1.0)
if msg and msg.arbitration_id == 0x481 and len(msg.data) >= 32:
    pos = int.from_bytes(msg.data[2:4], 'little', signed=True) # 0.01mm 整数
    print(f"位置: {pos/100:.2f} mm")
# can.py 接收路径：ReadFD 拿到 TPCANMsgFD(msgFD) → DLC=13 反查32字节 → bytes
# 应用层：按 xlsx"上位机接收"表取 [2:4] 小端解码
```

步骤4 失能

```
bus.send(can.Message(arbitration_id=0x201, data=bytes.fromhex('00000000000400'),
                    is_extended_id=False, is_fd=True))
```

时序全景

上位机	总线	驱动器 ID=1
ID=0x201 data=..0700	—————→	使能
ID=0x201 data=018813dc050f00	—————→	移动到50mm
ID=0x601 data=2B03180520..	—————→	配置32ms上报
←—————	ID=0x481 data=32字节状态	(周期自动发)
解析 data[2:4] → 位置		
ID=0x201 data=..0400	—————→	失能

第9章 FAQ：曾经真实困惑过的问题

Q1 `arbitration_id` 是这个项目发明的容易混淆的概念吗？

不是。它就是 CAN 标准里唯一的那个"仲裁ID"。这个英文命名沿用自业界最流行的 `python-can` 库（本项目 `can.py` 模仿它的接口），是通用行业惯例。项目私有的只是"往这个字段填什么数"的公式（`0x200+关节ID`）。

Q2 仲裁 ID 是几个字节？

标准帧的 ID 是 11 个比特（连1个字节都不到），存放在 `c_uint`（4字节）容器里，容器比内容大不代表数据有4字节。它和 data 的"7字节"没有任何关系——那是数据段的长度。

Q3 使能=00000000000700、位置=01+目标+时长+0F00，这个切分规则谁定的？出处在哪？

出处是代码本身（硬编码在 `main.py/calib_v3.py` 等），xlsx 没有文档化这个 0x200 通道格式。字段规律是通过并排对比 `positionControl/speedControl/currentControl/is_enable` 多处调用反推出来的（第1字节变=模式，中间4字节=目标+时长，尾2字节=控制字）。真正的官方出处大概率是厂商早期提供的示例代码或私下沟通，现有文档里查不到。

Q4 msgFD 是什么？

接收方向的原材料：`Bus.recv` 调 `ReadFD` 从硬件拿到的一帧，包在 `TPCANMsgFD` 结构体里，参数名叫 `msgFD`，交给 `Message(msgFD=...)` 反向翻译成 Python 对象。发送时你造结构体给驱动，接收时驱动造好结构体给你，同一种结构体、填的人不同。

Q5 感觉有 data1（Python里的）和 data2（CAN帧里的）两份数据？

只有一份。hex字符串 → Python bytes → C数组 `DATA[64]` 前几格 → 总线电信号，字节数值从头到尾没变，变的只是容器（见 5.4 节旅程图）。

Q6 为什么用 0x200+device_id 这种结构，不给每台设备随便分配固定 ID？

CANopen"预定义连接集"惯例：扩展方便（新设备配个号就能用）、固件实现极简（一行加法）、优先级天然分类（控制帧基础值小、总线拥挤时自动优先）。见 5.1 节。

Q7 为什么 can.py 要逐字节拷贝 data？

`DATA` 是 ctypes 定长数组 (`c_ubyte*64`)，与 Python bytes 内存布局不兼容，整体赋值要求长度相等（7≠64 报错），所以循环填前几格最稳妥。纯工程细节，与协议无关。

Q8 xlsx 到底在哪用上了？"使能+移动"的例子好像完全没用到？

对，0x200 控制通道 xlsx 没文档化，那个例子确实用不到 xlsx。xlsx 真正用上的地方：`SDO` 配置类（offset校准/改ID/力标零/上报/心跳，第7章标√的条目）和 0x480 反馈报文的逐字节解析（[2:4]取位置这个偏移就是抄 xlsx"上位机接收"表的）。完整对应看 7.0 速查总表最后一列。

Q9 仲裁段到底是"ID+RTR"还是"0x200+关节ID"？

不同层次：CAN 标准说的仲裁段是帧里用于优先级裁决的比特区域（ID+RRS等）；"0x200+关节ID"是应用层往 ID 字段填数的公式。0x201 既是标准意义的11位ID，也是协议意义的"发给1号关节的控制帧"，不矛盾。

Q10 能不能 pip install python-can？

不能。本项目模块名就是 `can.py`，装公共库会 import 冲突。

Q11 扩展帧（29位ID）用了吗？

没有。全部 `is_extended_id=False`，11 位标准帧。

Q12 docx 和 xlsx 是同一份协议吗？

不是。docx 是"关节"（旋转）产品线字典，xlsx 才是"推杆"（直线）协议，同索引含义可能不同，本项目一律以 xlsx 为准（见 6.1 节）。

附录A DLC 映射表（CAN-FD，CiA 规范）

实际字节数	DLC 值
0~8	0~8（相等）
9~12	9
13~16	10
17~20	11
21~24	12

实际字节数	DLC 值
25~32	13 (推杆32字节反馈走此档)
33~48	14 (48字节反馈走此档)
49~64	15

can.py 发送时正向查表（第39~55行），接收时按 DLC 反查真实长度（第72~89行）。

附录B SDO 命令字（cs）速查

CS	含义
0x23	写4字节
0x2B	写2字节
0x2F	写1字节
0x40	读请求
0x60	写成功响应
0x43/0x47/0x4B/0x4F	读成功响应（4/3/2/1字节有效数据）
0x80	失败（abort）

附录C 已知问题待办清单

1. main.py 的 Statusword6041 故障文字表已过时，应按 xlsx 0x6041 31位定义重写（7.15节）。
 2. calib_v3.py 第118行注释 index/subindex 写反（正确：0x2001:05），文档已更正，代码注释未改。
 3. calib_v4.py read_status() 借心跳读状态后未把周期改回0，多次调用会持续占用总线（7.8节）。
 4. xlsx 内部矛盾：“设置ID”在两个 sheet 里分别写 0x2033:04 和 0x2001:01，代码用后者且有效，建议与厂商确认。
 5. PVT 力位混合控制（0x500+id）与标准伺服状态机（SDO 0x2011）xlsx 有文档但代码未实现；故障后也没有自动清故障（0x2011:01=0x0035）逻辑。
 6. docx（关节）与 xlsx（推杆）易混用，查协议时先确认产品线，本项目固定以 xlsx 为准。
- *文档版本：V3（重构版）。协议结论已用 KAI执行器通信32字节协议.xlsx 原文逐条核对；如厂商更新协议，以最新 xlsx 为准。配套速查：CAN-报文对照表.md（本文档第7章的独立单行本）。*